

CONFIDENTIAL · INTERNAL REFERENCE



SARANG

BUSINESS OS LITE

Product, Architecture & Engineering Report

Business Decisions · System Design · AI Assistant · Feature Catalog · Full Business-Type Catalog

Aszurex — Trust Beyond Limits

Version 1.1.2 · 2026-07-23

Prepared as a complete, current, single-document technical and business record of Sarang's design and state

Table of Contents

CONFIDENTIAL INTERNAL REFERENCE

1	Executive Summary
2	Vision & Business Rationale
3	Market Opportunity & Strategic Framework
	3.1 The Roti, Kapda, Makan Framework
	3.2 Total Addressable Market
	3.3 Regional Targeting Logic
4	Why Choose Sarang — Competitive Positioning
	4.1 Head-to-Head Comparison
	4.2 The Real Gap Sarang Fills
	4.3 Where Sarang Is Honestly Not the Right Fit
5	Key Business Decisions Log
6	Development History — Phase Timeline
7	System Architecture
	7.1 High-Level Architecture & Process Boundary
	7.2 Technology Stack — And Why Each Choice Was Made
	7.3 Billing Transaction Data Flow
8	Database Design
9	Security & Data Integrity
10	AI Assistant — "Ask Your Business"
	10.1–10.2 Why It Exists & Model Selection Journey
	10.3 Query Pipeline Architecture
	10.4–10.5 Safety Guarantees & Performance
11	Feature & Module Catalog
12	Business Types Supported — Full Feature Catalog (43 Verticals)
13	Language Support
14	Screenshots — Real Application Screens
15	Testing, QA & Release Process
16	Distribution & Deployment
17	Cost & Investment Estimate
	17.1 Traditional-Equivalent Development Cost
	17.2 Actual Cost Incurred — Founder's Launch Budget
18	Honest Assessment & Known Limitations
19	Development Scale & Effort
20	Document Revision History

Executive Summary

CONFIDENTIAL · INTERNAL REFERENCE

Sarang Business OS Lite is an offline-first desktop business-management application built by Aszurex, a solo-founder software studio. It runs entirely on the user's own computer — billing, inventory, customers, suppliers, reporting, and an on-device AI assistant — with zero cloud dependency, zero subscription cost, and zero mandatory telemetry.

The product serves two deliberately stated purposes: help small and mid-sized businesses run their operations without recurring software costs, and demonstrate Aszurex's engineering capability to prospective clients who might need custom software, ERP, or CRM work. Sarang is, by design, both a free product and a portfolio asset — this dual purpose shapes several of the decisions documented in Section 5.

43

Business types supported out of the box, from retail and restaurants to clinics, law firms, and hotels — full feature catalog in Section 12.

13

Languages built in: 8 Indian languages plus Spanish, French, Portuguese, Indonesian, and Arabic (RTL).

185

Database models in the Prisma schema, covering every vertical's data without a server.

134,086

Lines of TypeScript/TSX across 594 source files (excluding tests, config, generated code).

1.03 GB

Current (v1.1.2) installer size — the direct, disclosed cost of bundling a local AI model rather than requiring internet access.

920 / 920

Live end-to-end checks passing across 44 suites, real Electron app, real database — see Section 15.

What this report covers

This document is a complete technical and business record of Sarang as it stands today, at **v1.1.2**: the reasoning behind its architecture, the real (including rejected) options considered for its AI subsystem, its data model, its full feature and business-type coverage (Section 12 catalogs every one of the 43 verticals' distinctive features individually — not just a name list), and how it is built, tested, and shipped. Every fact in this report is drawn directly from the project's own specification documents, source code, and schema — not reconstructed from memory or inferred after the fact. Where a decision was revised or reversed during development, both the original and the final position are recorded, because the reasoning behind a reversal is often more instructive than the decision itself.

On this document's format. Earlier working drafts of this report grew as a chain of dated addenda, each appended after an independent re-verification pass, preserving a running history of what was known at each point in time. This edition consolidates that entire history into one current, internally-consistent document — the full revision history is retained in Section 20 for anyone who wants the "as it was believed on date X" trail, but every number and claim in Sections 1–19 below reflects the current, most-recently-verified state, not a snapshot from partway through that history.

Vision & Business Rationale

CONFIDENTIAL - INTERNAL REFERENCE

2.1 Why Sarang Exists

Sarang's founding brief names two purposes explicitly, not one:

Purpose 1. Help businesses operate without recurring software costs — a genuinely free, complete operating system for day-to-day operations.

Purpose 2. Demonstrate the engineering capability of Aszurex to businesses that might need custom software built for them.

The intended arc for a user is stated as a single sentence in the project's own branding specification:

"If Aszurex built this for free, they can definitely build software for my company."

— docs/BRANDING_AND_MARKETING.md, Core Principle

2.2 Positioning

Sarang is deliberately positioned as "A Free Offline Business Operating System" — not as POS software, not as billing software, not as restaurant software, even though those are all things it does. The broader framing (operations, inventory, reporting, analytics, workflow, business intelligence foundations) is intentional: it keeps the product's ceiling high and avoids boxing it into a single-category tool in the eyes of a prospective client evaluating Aszurex.

2.3 Monetization Philosophy — And Its Evolution

The original monetization model was donation-based: Sarang is free, full stop, with an entirely optional, never-required, never-feature-gated donation path for grateful users. That rule set is strict — donations must never be tied to features, support, or updates, and must never be pursued aggressively.

That model was later revisited. The founder's own assessment concluded that a donation-based model does not scale as a real revenue path for continued development, and the monetization strategy shifted toward two channels that don't compromise the "free and offline forever" promise to end users: a future cloud / multi-branch waitlist (an optional paid tier for businesses that outgrow a single-install, single-location model) and a CA (Chartered Accountant) partner channel (referral/distribution partnerships with accounting professionals who serve exactly the small-business audience Sarang targets). Both preserve the core commitment: the free, offline, single-business product never becomes worse or more restricted to fund the business.

2.4 Trust & Transparency Commitments

These are treated as product requirements, not marketing copy — they are enforced in code (no default telemetry, no network calls from the core app or the AI subsystem) and restated verbatim across the app's UI:

"This software is completely free. No subscriptions. No monthly fees. No cloud dependency. No user tracking. No telemetry by default. No data collection by default. Your data stays on your device. Aszurex does not have

access to your business data."

— docs/BRANDING_AND_MARKETING.md, Transparency Section

CONFIDENTIAL · INTERNAL REFERENCE

SARANG BUSINESS OS LITE — PRODUCT & ENGINEERING REPORT Aszurex · Confidential

Market Opportunity & Strategic Framework

CONFIDENTIAL - INTERNAL REFERENCE

Sourced directly from the project's own market-research and founder planning documents (docs/SARANG_FOUNDER_NOTES.md), not reconstructed after the fact. Figures are the research inputs that shaped Sarang's design decisions — presented as the evidence the founder actually worked from, not independently re-verified by this report.

3.1 The Roti, Kapda, Makan Framework — Why These Verticals, In This Order

Sarang's business-type prioritization follows an explicit organizing principle, not an arbitrary list: India's (and most of the world's) informal economy runs on three basic-needs sectors — **Roti** (food: restaurants, dhabas, bakeries, grocery), **Kapda** (clothing: garment retail, textile wholesale, manufacturing/export), and **Makan** (shelter: construction materials, hardware, contractors, real estate). Each sector was evaluated independently — what's built, what's a known gap, and its real market size — before being sequenced into the phase roadmap (Section 6).

Sector	Market Size (India)	Sarang's Coverage
Roti (Food)	Millions of food-service MSMEs; a "complete ecosystem" across restaurant, grocery, bakery, fish/seafood, and dhaba business shapes	Built — Restaurant vertical is the single most feature-dense in the app (Section 12.1)
Kapda (Textile)	\$174B (2024) → \$350B by 2030, 45 million employed, 80%+ MSMEs	Built at the retail-garment layer (Clothing/Footwear verticals); wholesale and export-manufacturer layers remain a documented future scope, not yet shipped
Makan (Shelter)	India construction sector ₹6.34 trillion (2025), CAGR 8.1%; 2M+ small contractors (₹50L–₹5Cr revenue)	Built at the materials-retail layer (Hardware vertical); dedicated contractor/BOQ billing remains a documented future scope

Honest framing, consistent with this report's standing practice: this framework explains *why* these sectors were chosen as the product's spine, not a claim that every layer of each sector is already built. Section 12's catalog describes only what's actually shipped in the 43 live business types — where this framework's aspirational scope (e.g. a dedicated textile-manufacturer/exporter template) hasn't been built yet, it is not counted in this report's feature claims.

3.2 Total Addressable Market

Level	Definition	Size
TAM	All MSMEs globally that could benefit from a business OS	500+ million businesses worldwide
SAM	MSMEs with a computer/laptop, some digital literacy, in covered industries	~8–10 million (India); ~30 million (global)
SOM	Realistic first-3-year target: South India + Hindi belt MSMEs	500,000 active installs (target, not yet reached — see Section 18.2 on external validation)

India's MSME base: 63 million+ registered (Udyam portal, 2025); only ~29% of tech-aware MSMEs use any accounting software today, with the rest on Excel, khata books, or nothing at all — 18% cite high software

cost as the primary reason. India's business-software market itself is sized at USD 16.54B (2025), projected to reach USD 33.72B by 2032 (10.76% CAGR). 65% of MSMEs surveyed prefer simple, customizable tools over complex ERPs — directly informing Sarang's UX mandate (Section 11.3).

3.3 Regional Targeting Logic

Sarang's language rollout (Section 13) was not sequenced arbitrarily — it follows documented regional MSME density and sector concentration data. Representative examples from the research: **Gujarat** (5.7M+ MSMEs, highest per-capita business formation rate in India; Surat's diamond-cutting industry alone is a \$24B export sector with 500,000+ small-unit workers) and **Tamil Nadu** (4.2M+ MSMEs; Tirupur's knitwear cluster is a \$4.5B export hub with 10,000+ garment units) were prioritized specifically because their language (Gujarati, Tamil) and sector needs (textile/apparel billing) map directly onto verticals Sarang already builds for. This is the documented reasoning behind shipping Gujarati and Tamil in the very first language set, ahead of larger-population languages that don't map to as concentrated a business use case.

Why Choose Sarang — Competitive Positioning

CONFIDENTIAL - INTERNAL REFERENCE

This section states Sarang's real value proposition against real, named alternatives — with honest limitations included, consistent with this report's standing refusal to write pure marketing copy (Section 18 exists specifically because this document doesn't do that). Pricing and feature claims for competitors are drawn from the project's own market-research documents and are approximate, publicly-known figures, not independently re-verified by this audit.

4.1 Head-to-Head Comparison

Product	Model	Strength	Weakness	Price
Tally Silver	One-time desktop	Every CA knows it, 40-year market moat	Old UI, no mobile, ongoing renewal cost	₹18,000 + ₹4,500/yr
Vyapar	Freemium mobile+desktop	1 crore+ users, WhatsApp sharing, Hindi	Accounting-first, no industry depth, no Kannada/Tamil	Free + ₹2,499/yr
Zoho Books	Cloud subscription	40+ integrated apps, GST/e-invoice	Needs internet, data stored on Zoho's servers	₹3,999/yr
Busy Accounting	Desktop	Deep textile/pharma industry features	Dated UI, no South Indian languages	₹9,000–₹18,000
Marg ERP	Desktop	Pharma/FMCG distribution depth	Narrow industry focus, poor UX, no regional languages	₹10,000+
ERPNext	Open source / cloud	Free, e-invoice, 150+ countries	Needs a server — not truly offline	Free + hosting cost
Khatabook / OkCredit	Free mobile apps	Combined 55M+ users — bigger than Tally's entire base	Credit-ledger only — no real invoicing, no inventory, cloud-synced to their own servers	Free
Sarang	Free offline desktop	43 business types, on-device AI, zero data collection, modern UI	No cloud multi-branch sync (v1); zero external validation to date (Section 18.2)	Free forever

4.2 The Real Gap Sarang Fills

No product identified in this research is simultaneously:

- Truly offline — zero cloud dependency, not "cloud with an offline mode"
- Free forever for everyone globally — no feature-gating, no geographic price discrimination
- Industry-template driven — one shared engine, 43 verticals, not a generic form relabeled per industry
- Privacy-absolute — zero data collection by architecture, not by policy that could change
- Built with a modern, non-technical-user-friendly UI (Section 11.3's 16px/44px/48px mandate)

Khatabook and OkCredit matter specifically because of scale, not feature depth: combined, they have more users than Tally. They solve one narrow problem (credit/khata tracking) that Sarang also solves, as one module among many — meaning any of their combined 55M+ users doing real invoicing and inventory by

hand alongside a credit-tracking app is a direct, better-fit candidate for Sarang, not a competitor's customer to win over from scratch.

CONFIDENTIAL · INTERNAL REFERENCE

4.3 Where Sarang Is Honestly Not the Right Fit

Stated plainly, matching this document's practice everywhere else:

- Businesses that need real-time cloud sync across multiple physical locations today — Sarang is deliberately single-install, single-business in v1 (Section 5's foundational decision); multi-branch is a planned future paid tier, not a v1 feature.
- Businesses that need a certified, audit-grade accounting/P&L statement for statutory filing — Sarang is explicitly a business-operations tool, not accounting software (Section 9.1's reports are operational references; a CA should still produce certified statements).
- Businesses that need proven mobile/WhatsApp-first workflows as the primary interface — Sarang is desktop-first; there is no mobile companion app in the current shipped product.

4.4 The Single-Sentence Pitch

"You're using Khatabook for credit and writing invoices by hand — Sarang replaces both, works offline, and your data stays with you."

— the project's own stated market pitch, docs/SARANG_FOUNDER_NOTES.md

SECTION 5

Key Business Decisions Log

CONFIDENTIAL - INTERNAL REFERENCE

The decisions below are the ones that most shaped what Sarang became — including several that were reversed mid-project after new evidence. Each is recorded with its actual reasoning, not just its outcome.

Foundational — Offline-first, no cloud dependency, as the top design principle

Internet access is optional for every core operation. No cloud servers, no forced telemetry, no customer/business data transmission of any kind by default. This single principle downstream-determined the choice of Electron, SQLite, and later the entire AI-assistant safety architecture.

Foundational — Single-business, single-install model

Sarang deliberately does not support cross-install networking or a multi-branch data model in v1. This constrains some designs (e.g. referral fields between records are free text rather than foreign keys, since there is no second install to link to) and is the reason a future multi-branch capability is treated as a separate, paid product tier rather than a v1 feature.

2026-07-03 — "No AI" was revised from a blanket rule to a data-transmission rule

Sarang's original non-negotiables list included "no AI" alongside "no cloud, no telemetry, no tracking." That rule was reinterpreted: the actual commitment was always zero data transmission, not zero local computation. A model running entirely on-device with zero network calls does not violate the original intent — "no cloud AI" became the operative rule.

2026-07-03 — The AI model ships bundled in the installer, never downloaded on first run

Explicit founder instruction, preserved verbatim in the spec: "once we have downloaded Sarang, internet should never be required — if that's true, proceed even if it's 2GB." This overrode the installer's original <150MB size target.

2026-07-13 — AI model pin changed after real benchmarking, twice

Phi-3.5-mini-instruct (the original pin) was benchmarked and rejected on real hardware performance. Qwen2.5-3B-Instruct was rejected on licensing grounds. Qwen2.5-1.5B-Instruct (genuinely Apache 2.0) became the final pin. Full detail in Section 10.2.

2026-07-13 — The AI model never generates or executes SQL

Described in the project's own spec as "the central safety decision of this phase." Full detail in Section 10.3–10.4.

Ongoing — No code signing for Windows in this release

The installer ships unsigned; the build config supports signing via environment variables the moment a certificate is available. Addressed directly with the founder on 2026-07-22 (three concrete signing options costed out: Azure Trusted Signing, a traditional certificate, an EV certificate) — the founder's considered decision remains to decline for now, not an oversight. Still true as of v1.1.2.

Ongoing — No auto-update mechanism

`publish: null` in the packaging config — a deliberate choice for a fully offline, no-telemetry product. The accepted cost is that every future patch reships the entire multi-gigabyte installer.

2026-07-16 — Distribution via GitHub Releases, not a self-hosted download server

The installer is hosted as a versioned GitHub Release asset, linked from a dedicated product page on the Aszurex website via a fixed-name asset (Sarang-Setup-latest.exe) that keeps the public download link stable across every future release without any website changes — see Section 16.

2026-07-21/22 — Systemic timezone and AI-cache bugs, and a 65-route coverage gap, found and closed

An extended work session found and fixed four real, previously-undetected defects (see Section 6), the most severe being a UTC-vs-local date-boundary bug affecting reports and the AI Assistant across the entire application. A permanent 139-check full-screen-crawl test suite was added afterward specifically because the same session found 65 of 123 real screens had never been visited by any automated check.

2026-07-23 — A second, independent audit round was commissioned the same day the first was reported complete

Rather than treat one thorough audit as sufficient, the founder requested an entirely fresh pass — "no prior context carried forward" — the same day the first was reported done. It found 15 more real defects the first pass missed, including a CRITICAL migration-runner bug from an earlier, separately-run audit that this project's own primary audit-log document didn't know had already been fixed. The concrete lesson this produced, and the reason it's recorded as a business decision rather than just an engineering note: **this project now treats "audit complete" as a claim that itself needs re-verification, not a terminal state** — see Section 18 and Section 20.

Development History — Phase Timeline

CONFIDENTIAL · INTERNAL REFERENCE

Sarang was built as a continuous sequence of numbered phases, each following the same spec-first, approval-gated process: audit against live code, written technical spec, sign-off, implementation, tests, independent verification, self-audit, completion report. No phase was declared complete on the strength of "the code compiles" — every completion report required a re-verified test count and, for anything touching money or permissions, live UAT against the real application.

Era	Phases	What Shipped
Foundation	1–21	Core billing, inventory, customers/suppliers, reports, roles/permissions, backup — the original 9 business types plus Manufacturing.
V2 Service Platform Expansion	22–38	24 new service-business verticals, the TemplateModule gating system, barcode generation/printing, loose/weight-based billing, and a customer-facing QR ordering page served over the local LAN.
Product Hardening	39–56	Branding, UX polish, 13-language localization, in-app Manual + PDF export, additional verticals (Hotel/Lodge, Diagnostic Lab, Blood Bank, Rental, Jewellery), international currency/tax generalization, and a large expansion to 70 additional AI-drafted business templates validated by live UAT.
AI Assistant	57	The on-device "Ask Your Business" subsystem — sequenced deliberately behind the hardening bundle, per an explicit founder decision to validate the core product with real users before adding AI.
Full Vertical Coverage & Restaurant Binding	58	All 43 business types brought to full parity across all 13 languages; restaurant table-merge, split-bill, and reservations built on a real table↔order data-binding gap discovered and fixed mid-design. Completed 2026-07-21.
Post-Release Hardening (two rounds)	—	2026-07-21/22: offline password recovery, bargained pricing, a systemic timezone bug closed across 15 files, an AI stale-cache bug fixed, a 65-route test-coverage gap closed. 2026-07-23: a second, independent from-scratch audit found 15 more real defects the first round missed (Section 18), followed by the v1.1.2 release itself (Section 16).

A representative bug story

Every printed invoice and report once displayed currency with no digit grouping at all — ₹14568.00 instead of ₹14,568.00 — because the print/export subsystem had its own, separate, naive number formatter rather than reusing the app's correct one. It was found only during a full live-verification pass of the packaged installer, not by any automated test, and required fixing 48 separate call sites, 9 print templates, and one external caller. This kind of finding — a real defect that unit tests didn't catch because the bug was in formatting, not logic — is representative of why this project treats "0 TypeScript errors, all tests passing" as necessary but never sufficient for a release.

A second, structurally similar story from 2026-07-23: a systemic date-range fix believed "fully closed" across ~20 files in one audit round turned out to have only fixed the *start* of each range — the *end* bound was still silently wrong for any user not in a positive UTC offset, on ~38 more call sites, undetected by the very audit that believed it had closed the bug class entirely. Full detail in Section 18.

System Architecture

CONFIDENTIAL · INTERNAL REFERENCE

7.1 High-Level Architecture & Process Boundary

Sarang is a two-process Electron application: a sandboxed renderer process (the React UI, with no direct Node.js or filesystem access) and a privileged main process (Node.js, holding the only Prisma/SQLite connection). The two communicate exclusively through a typed, validated IPC bridge — the renderer never touches the database, the filesystem, or any Node API directly.

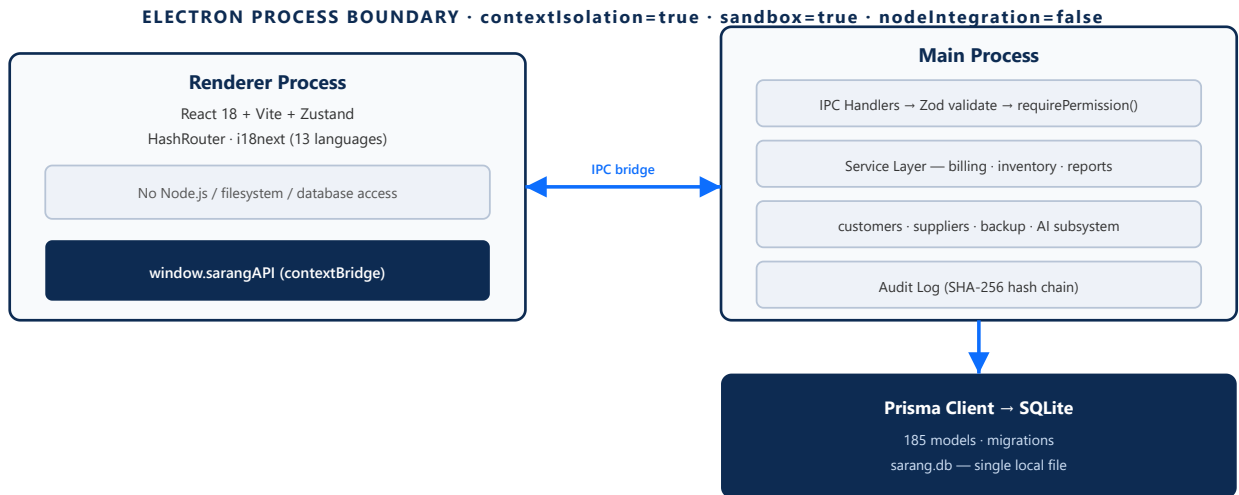


Figure 7.1 — Renderer and main process are separate, sandboxed OS processes; every data path crosses the IPC boundary once.

7.2 Technology Stack — And Why Each Choice Was Made

Reasoning below is drawn directly from the project's own architecture specification (`docs/ARCHITECTURE.md`), not reconstructed after the fact.

Layer	Technology	Version	Why Chosen
Desktop shell	Electron	33.3.1	Mature, cross-platform, offline-friendly, largest ecosystem of any desktop-app runtime — directly serves the offline-first architectural principle (Section 5)
UI framework	React + TypeScript	18.3.1 / 5.7.3	Maintainability and scalability across a 43-vertical UI; type safety catches an entire class of bugs before runtime, critical at this codebase's scale (134,086 lines)
Build tooling	Vite / electron-vite	6.0.7 / 3.1.0	Fast renderer + main process bundling and dev server, purpose-built for the Electron dual-process model
Packaging	electron-builder	26.15.3	NSIS installer, asar packing, native-module unpacking — the de facto standard for production Electron distribution
State management	Zustand	5.0.3	Minimal boilerplate renderer-side stores, avoiding Redux-scale ceremony for a desktop app's actual state needs

Data tables/lists	TanStack Table / Virtual	8.21.2 / 3.14.3	Virtualized rendering keeps large invoice/inventory lists fast regardless of row count
Forms & validation	React Hook Form + Zod	7.54.2 / 3.24.1	One schema definition validates both the form UI and the IPC payload boundary — a single source of truth instead of two independently-maintained validators
Database	SQLite via Prisma	6.2.1	Embedded, no server required, reliable, fast, trivially backed up as a single file — directly serves the local-first principle; Prisma adds type safety, migrations, and maintainability over raw SQL
On-device AI	node-llama-cpp	3.19.0	Local GGUF model inference with zero network calls — the only way to satisfy the "zero data transmission" AI safety requirement (Section 10.4)
Internationalization	i18next / react-i18next	26.3.1 / 17.0.8	Mature 13-language runtime translation with RTL support, all locale files bundled — no network fetch ever, matching the offline-first principle
Styling	Tailwind CSS	3.4.17	Small bundle size, fast development, visual consistency across 43 verticals' worth of screens without a custom design-token system
Charts	Recharts	2.15.0	Dashboard KPI and report visualizations rendered entirely client-side, no external charting service
Password hashing	bcryptjs	2.4.3	Pure-JS — no native compile step for auth
Testing	Vitest / Playwright-core	4.1.9 / 1.61.1	Unit tests; real-Electron end-to-end UAT harness

7.3 Billing Transaction Data Flow

A single invoice creation touches permissions, inventory, ledgers, and the tamper-evident audit log in one guarded pass: Billing UI → IPC + Zod validate → `requirePermission` → `billing.createInvoice` → one Prisma transaction (Invoice + Items + Payment + Inventory + Ledger) → SQLite write → AuditLog hash-chain entry. Every write is wrapped in one Prisma transaction; a broken hash chain on any AuditLog row is detectable on next verification.

Electron security baseline

Renderer isolation: `nodeIntegration: false`, `contextIsolation: true`, `sandbox: true` — the renderer cannot reach Node, the filesystem, or Prisma directly under any circumstance. **Bridge surface:** all capability is exposed through a single `contextBridge` API (`window.sarangAPI`), with `webviewTag: false` and `enableRemoteModule: false` closing off legacy escape hatches. Independently re-verified across every audit round in this document's history (Sections 18 and 20), including this session — every `BrowserWindow` in the app (main window, splash, kitchen-display window, and every ad-hoc print/export/PDF window) consistently applies this same configuration.

Database Design

CONFIDENTIAL - INTERNAL REFERENCE

Sarang's entire data layer is a single embedded SQLite file (`sarang.db`) managed through Prisma — **185 models** in total (directly counted, `grep -c "^model " prisma/schema.prisma`), spanning identity, commerce, and 30+ vertical-specific domains, with zero external database server required at install or runtime.

8.1 Domain Overview

Domain	Representative Models
Identity & Access	BusinessProfile, User, Role, Permission, RolePermission
Catalog & Inventory	ProductCategory, Product, Inventory, InventoryMovement, ProductVariant, ProductBatch, ProductSerial
Billing & Money	Invoice, InvoiceItem, Payment, CreditNote, DebitNote, Quotation, QuotationItem
Parties & Ledgers	Customer, CustomerLedger, Supplier, SupplierLedger, PurchaseOrder, PurchaseOrderItem
Operations & Compliance	AuditLog, Setting, TaxConfiguration, Backup, Notification
Restaurant vertical	RestaurantTable, KOT (kitchen order ticket), Recipe
Manufacturing vertical	RawMaterial, BillOfMaterial, ProductionOrder
Hotel / Lodge vertical	HotelRoom, HotelBooking, HotelGuestId
Rental vertical	RentalUnit, RentalBooking
Clinical / Lab verticals	Appointment, Pet, VaccinationRecord, LabTestOrder, NormalRangeReference
Blood Bank vertical	Donor, BloodIssue
Logistics vertical	Shipment, GRN, DeliveryChallan
Jewellery vertical	MetalRate, MetalExchange
AI subsystem	AiQueryLog (local-only, never transmitted)

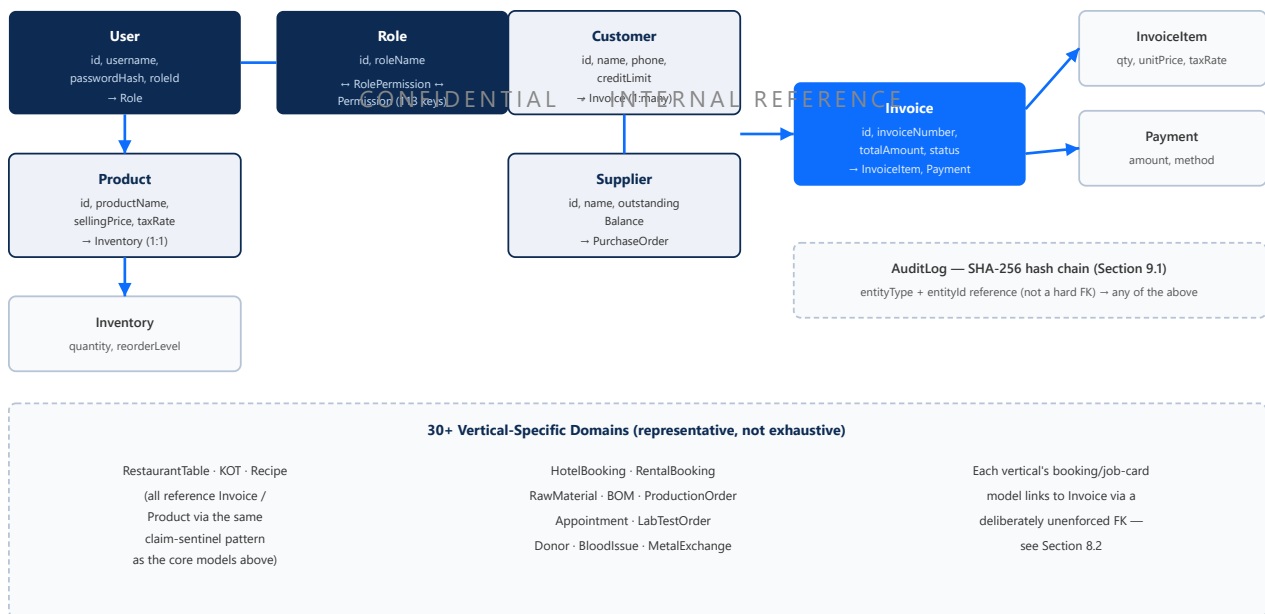


Figure 8.1 — Core domain model (representative subset of 185 total models; solid boxes are strict foreign keys, dashed boxes use the deliberate reference-not-FK pattern explained in Section 8.2).

8.2 Notable Schema Design Decisions

No cross-install foreign keys. Because Sarang is single-business/offline with no cross-install networking, referral-style fields (e.g. a lab referral) are deliberately free text rather than foreign keys into another install's data.

Computed, not stored, availability. Rental availability is derived live from active bookings rather than a stored inventory count, and "OVERDUE" is a derived status, never a stored one — avoiding a whole class of stale-state bugs.

Hotel/Lodge is a separate vertical from Rental, deliberately. Legal guest-register requirements, per-night billing, and in-stay folio charges are different enough in shape from general rental that the schema treats them as distinct domains.

Guest ID validity is enforced at the application layer, not the schema. Valid ID-document types depend on the business's own country, which a single SQLite column can't itself encode as a constraint.

Deliberate, documented "unenforced" FKs on ~20 vertical booking/job models. Fields like `invoiceId` on RentalBooking, HotelBooking, ServiceTicket, JobCard, DrivingSession, etc. are not database-level foreign keys — this is the atomic claim-sentinel pattern behind the app's double-invoice-race protection (Section 18), documented in-schema, not an oversight.

Security & Data Integrity

CONFIDENTIAL · INTERNAL REFERENCE

9.1 Tamper-Evident Audit Log

Every audit-log entry is chained to the one before it using a SHA-256 hash: each row stores `prevHash` (the hash of the previous row) and its own `hash` (computed over its own data plus `prevHash`). A verification pass walks the real `prevHash` → `hash` links and reports either a clean chain or the exact row where a `hash_mismatch` or `chain_break` occurs. Concurrent writes are handled by serializing on the single "chain tip" setting row, so two simultaneous audit events can never both compute a hash against the same stale parent.

9.2 Backup Integrity Verification

Every backup is a full copy of the raw SQLite database file (via `VACUUM INTO`, after a WAL checkpoint) plus a manifest recording its SHA-256 checksum. Restoring a backup means extracting the file, recomputing its SHA-256, and comparing it against the manifest's recorded checksum before any data is touched. This same technique now also protects the mandatory pre-upgrade backup taken before a schema migration runs (a real defect where the pre-upgrade path used to be a weaker raw file copy was found and fixed — Section 18.3).

9.3 Permission Enforcement, Both Directions

Every screen and every IPC handler independently checks `requirePermission()` — permission is enforced server-side in the main process, not just hidden in the renderer's UI. Verified in both directions across multiple audit rounds: a Cashier-role account cannot reach Manager/Admin-only capability even via direct IPC calls that bypass the UI entirely, and an Admin account retains full access.

9.4 File-Path Handling

A real vulnerability was found and fixed this cycle: `BusinessProfile.logoPath` was accepted from the renderer as an unvalidated string, which could have allowed an arbitrary-file-delete (via the app's own unlink-on-change logic) and unescaped-HTML injection into every printed document. Fixed with a dedicated containment check (`isValidLogoPath()`) requiring the path resolve inside the app's own managed logo directory, plus defense-in-depth HTML-escaping on the generated file URL. Every other file-path handling surface in the app (backup/restore, document attachments, CSV import, the three LAN-facing HTTP servers) was independently audited this cycle and found sound — see Section 18.2.

AI Assistant — "Ask Your Business"

CONFIDENTIAL · INTERNAL REFERENCE

10.1 Why an On-Device AI Exists At All

The AI Assistant was not part of Sarang's original scope — "no AI" was on the initial non-negotiables list. It was added only after the founder reinterpreted that rule (Section 5): the actual commitment was zero data transmission, not zero local computation.

10.2 Model Selection Journey

Three models were evaluated in sequence, on real (not virtualized) representative hardware — an HP Pavilion laptop, Intel i5-1155G7, 15.8GB RAM.

Model	License	Result	Verdict
Qwen2.5-3B-Instruct	Non-commercial research license (not Apache 2.0 as first assumed)	Rejected on licensing grounds before benchmarking	REJECTED
Phi-3.5-mini-instruct	MIT (verified)	0.05–0.6 tok/sec · 5.1GB peak RAM · GPU auto-detect crashed with an out-of-VRAM error on integrated graphics	REJECTED
Qwen2.5-1.5B-Instruct	Apache 2.0 (verified)	~2.2 tok/sec · 1.8GB peak RAM · 27.7s cold / 5.8s warm pipeline latency	SHIPPED

Quantization: GGUF Q4_K_M, community-quantized (bartowski), ~941MB file. `gpu: false` is set explicitly in code — never left to library auto-detection.

10.3 Query Pipeline Architecture

The model never writes or executes SQL, and never touches the database directly — described in the project's own spec as "the central safety decision of this phase," a deliberate departure from an earlier draft architecture that had the model generate SQL directly.

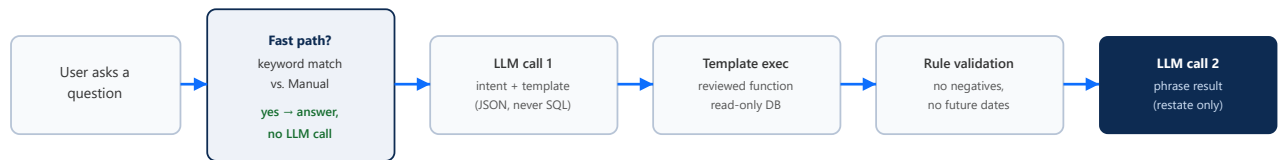


Figure 10.1 — Two-call pipeline: the model classifies and phrases; a reviewed, read-only function computes. The model itself never queries the database.

Answer plus a local-only AuditLog entry are the pipeline's final output — nothing beyond that leaves the two LLM calls' own reasoning, and neither call ever touches the network.

10.4 Safety Guarantees

No SQL from the model — enforced by the two-call architecture and tested by attempting to smuggle SQL-like text through intent extraction. **No advice, via hardcoded refusal** — legal, tax, medical, investment, and compliance questions are caught by a fixed `out_of_scope` intent category. **No fabrication** — a query returning zero rows or a null aggregate triggers a fixed fallback string before the phrasing call is even invoked. **Zero network calls, proven** — an automated test runs AI queries with network access blocked and confirms identical behavior.

10.5 Performance & Packaging

Metric	Value
Peak RAM (bounded 4096-token context)	1.8 GB
Generation speed	~2.2 tokens/sec
Cold pipeline latency	27.7 s
Warm pipeline latency	5.8 s
Model file size (GGUF Q4_K_M)	~941 MB
Real additive installer cost	~1.03 GB

`node-llama-cpp v3` is ESM-only, while Sarang's main process compiles to CommonJS — every call site uses a dynamic `await import('node-llama-cpp')`, which also lazy-loads the model on first use rather than at app startup. No third-party model or library name is ever surfaced in the user interface — to the end user, it is simply "Sarang AI Assistant." Routing accuracy on a 109-question live battery reached 100% after a targeted fix round (up from an initially-measured 89.9%); a separate stale-cache bug (all 6 KPI-backed templates reading a 60-second dashboard cache instead of forcing a fresh read) was found and fixed independently of that routing work.

Feature & Module Catalog

CONFIDENTIAL - INTERNAL REFERENCE

Every business type is built from a shared core plus opt-in modules, gated by an 88-flag TemplateModule system and a 113-key permission model — a business only ever sees the fields, screens, and reports relevant to how it actually operates. Section 12 catalogs exactly which modules each of the 43 business types enables.

11.1 Core Modules (present for every business type)

Module	What It Covers
Billing & Invoicing	Barcode support, loose/weight-based billing, GST-ready invoices, quotations, credit/debit notes
Inventory	Real-time stock, batch/serial tracking where relevant, reorder alerts, supplier linkage
Customers & Suppliers	Ledgers, outstanding balances, credit-limit enforcement, aging buckets
Reports & Analytics	Sales, inventory, tax (GST + generalized international tax), P&L, cash book, trial balance
AI Assistant	"Ask Your Business" — opt-in, off by default per business type
Backup & Restore	One-click local backup with integrity verification before every restore

11.2 Role-Based Access

Five seeded roles cover the realistic staffing shape of a small business: Admin, Manager, Cashier, Staff, Kitchen Staff — permission checks happen both in the UI and, server-side, inside every IPC handler as well, closing the common failure mode where a check exists only in the UI layer and a determined or buggy caller can route around it (independently verified in both directions across multiple audit rounds — Section 18.3).

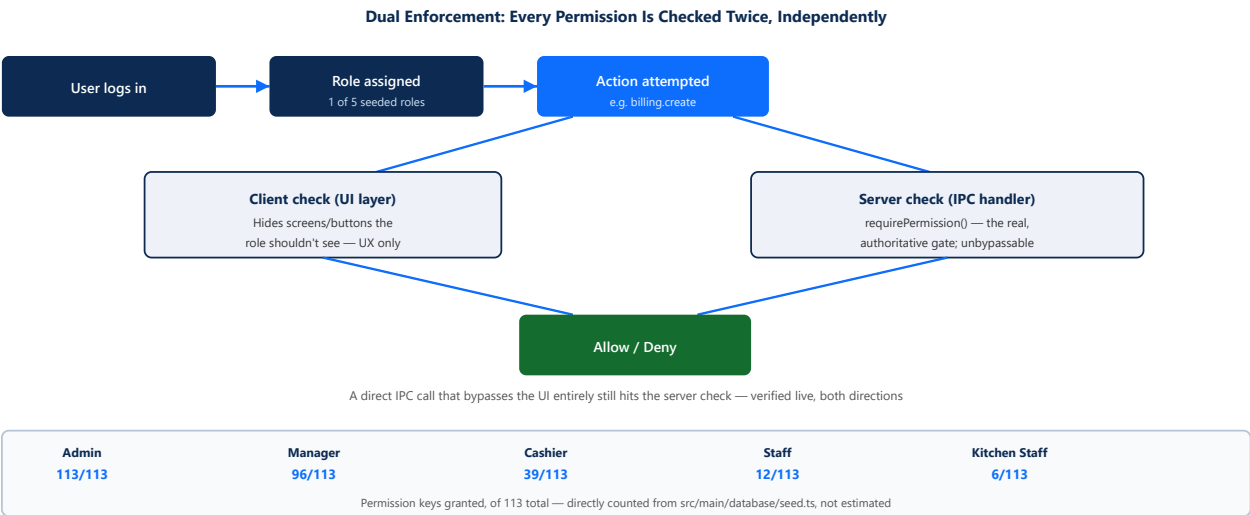


Figure 11.1 — Every permission-gated action is checked independently on both the client (UX convenience) and the server (the real, unbypassable gate) — verified live in both directions.

11.3 UX Design Philosophy

Sarang's interface follows an explicit "usable by an untrained small-shop owner" mandate: 16px base font (legible without zooming), 44px+ buttons (comfortable touch/click targets), 48px inputs (consistent form-field sizing throughout). Plain language is preferred over technical terminology everywhere a user-facing label appears, and no underlying technology name (React, Electron, Prisma, SQLite, or any AI model/library name) is ever surfaced anywhere a user can see it.

11.4 In-App Documentation System

A full in-app Manual is built into the product itself, available in all 13 supported languages with a matching PDF export. The AI Assistant's "navigation help" capability searches this same Manual content client-side by keyword — it is explicitly not sent to the AI model.

Business Types Supported — Full Feature Catalog

CONFIDENTIAL · INTERNAL REFERENCE

Compiled directly from `src/main/services/industry-template.service.ts`'s `TEMPLATE_DEFAULTS` map — the actual, live, single source of truth the running application uses to decide which screens and features a business sees, not a separately-maintained marketing description that could drift from what the code does. Every business type shares the same core (billing, inventory, customers/suppliers, reports, AI Assistant, backup — Section 11.1); what's listed below per vertical is what's *additional* and *distinctive* to it. All 24 "service-family" verticals (marked ♦) also share four common base modules — appointments, service catalog, provider schedule, notification queue — not re-listed per row to avoid repetition.

On "how useful is each one" — answered honestly, not uniformly. Every vertical below has live end-to-end UAT coverage confirming its distinctive screens load and its core workflow completes (Section 15; the 44-suite regression exercises all 43). That is a real, verified floor — but it answers "does it work," not "does anyone need it." Section 18.4 names four verticals added with no demonstrated external user demand behind them: **Blood Bank, Diagnostic Lab, Rental, and Agricultural Inputs**, marked † below. This is restated here rather than smoothed over — not because they are built any less solidly than the rest (they aren't; the same audit and test discipline applies to all 43), but because "well-built" and "wanted" are different claims.

12.1 Retail & Trade (14)

Business Type	Distinctive Features	What Makes It Different
Restaurant / Café / Food	Table management, Kitchen Order Tickets (KOT), recipes, ingredient-level stock tracking, table-merge/split-bill/reservations, kitchen display board, QR self-ordering	The most feature-dense vertical in the app — a full front-of-house + back-of-house workflow.
Retail / General Store	Returns, optional logistics (fleet/GRN/freight)	The baseline vertical everything else is compared against.
Hardware / Glass / Plywood	Area-based pricing (sq ft/sq m), credit-limit enforcement, logistics	Glass/plywood/tile is sold by area, not by piece — a generic SKU model can't price that correctly.
Distributor / Wholesale	Credit-limit enforcement, bulk order entry, outstanding-balance analytics, field order capture, logistics	Built for a sales-rep-in-the-field workflow rather than walk-in retail.
Pharmacy / Medical Store	Batch tracking, expiry tracking, logistics	Expiry-date compliance enforced as a first-class field, not an afterthought.
Electronics / Mobile Store	Serial number tracking, IMEI tracking, warranty tracking, repair/RMA workflow, logistics	Handles the full device lifecycle a generic product model has no vocabulary for.
Clothing / Textile / Apparel	Size/color variant tracking, returns, logistics	One product, many sizes/colors, one stock count each.
Footwear / Shoe Store	Variant tracking, returns, logistics	Same mechanism as Clothing, applied to shoe sizing.

Agricultural Inputs & Equipment †	Batch/expiry tracking (fertilizer/pesticide), serial/warranty tracking (equipment), job cards (equipment servicing), credit-limit enforcement, outstanding analytics, a dedicated agri dashboard, logistics	Combines Pharmacy-shaped compliance tracking with Electronics-shaped equipment tracking — a genuinely distinct real-world business shape.
Jewellery	Live metal-rate pricing (net weight × today's rate + making charge), returns, logistics, old-metal exchange against a new purchase	Sale price moves with the daily gold/silver rate — no other vertical prices this way.
Rental Business †	Checkout/return booking lifecycle, live-computed availability, logistics	Availability derived from active bookings in real time, closing a whole class of stale-state bugs by design.
Hotel / Lodge	Multi-night room booking, legal guest-ID register, per-night billing, in-stay folio charges, housekeeping	Kept separate from Rental — guest-register compliance and per-night billing are legally/operationally different.
Manufacturing / Production	Raw materials, Bill of Materials, production orders + analytics, work orders, dispatch tracking, finished goods, vendor management, logistics	The only vertical modeling a full production pipeline, not just buying and reselling stock.
General Business	Logistics only (opt-in)	The deliberate blank-slate option for a business that doesn't fit any specific template.

12.2 Clinical & Healthcare (7) ◆

Business Type	Distinctive Features	What Makes It Different
Veterinary Clinic	Pet/patient profiles, visit notes (SOAP-style), walk-in token queue	Records tied to a pet, not a human patient.
GP / General Physician	Visit notes, token queue	The baseline clinical vertical.
Specialist Clinic	Visit notes with referral-in/referral-out fields, token queue	Adds referral paperwork a specialist practice specifically needs.
Dental Clinic	Tooth-by-tooth dental chart, recall/follow-up scheduling, token queue	A visual, per-tooth charting model.
Physiotherapy Clinic	Visit notes, physio-specific treatment notes, session packs (pre-paid multi-visit bundles), token queue	Models physio as a course of visits, not one-off appointments.
Diagnostic & Pathology Lab †	Test/panel order intake, result entry against normal-range references, token queue	The only clinical vertical modeling a test-order-then-result workflow.
Blood Bank †	Donor registration with cooldown enforcement, donation recording, screening, unit issue/stock	Safety-critical by nature — donor cooldown and screening-before-issue gating are hard constraints.

12.3 Wellness (3) ◆

Business Type	Distinctive Features	What Makes It Different
Beauty Salon / Spa	Session packs, staff commission tracking, multi-service booking	Models a real salon visit — several services in one sitting.
Gym / Fitness Studio	Session packs, memberships (atomic session-cap enforcement), batch/group classes, staff commission	Membership session-cap enforcement was a real race-condition bug found and fixed this cycle — now closed transactionally.
Driving School	Session packs, learner profiles, driving-session scheduling with package-cap enforcement	Package limits now genuinely enforced server-side, not just filtered in the UI.

12.4 Professional Services (8) ◆

Business Type	Distinctive Features	What Makes It Different
Lawyer / Law Firm	Legal case management, billable time entries	Case-centric record model.
CA / Chartered Accountant	Compliance task tracking, client engagements, time entries	Engagement-centric, with statutory-deadline compliance as a first-class object.
Company Secretary	Compliance tasks, ROC filings, board meeting records, time entries	The only vertical with board-meeting/statutory-filing record-keeping.
Architect	Leads, service projects, time entries, drawing register	Drawing register tracks which drawing revisions were issued when.
Civil Engineer	Leads, service projects, time entries, site visit log	Site visit log is the civil-engineering-specific differentiator from Architect.
Real Estate Agent	Leads, property listings, commission-based deal invoicing	Deal/commission-centric rather than time-billed.
Consultant (legacy generic)	Projects, project tasks, work tracking, customer history	An older, simpler professional-services template, still actively maintained.
Consultant / Freelancer	Leads, service projects, retainers, time entries	Adds retainer billing for a solo professional's actual billing pattern.

12.5 Creative & Agencies (5) ◆

Business Type	Distinctive Features	What Makes It Different
Marketing Agency	Leads, service projects, retainers, campaign tracking	Campaign tracking is the marketing-specific addition.
Software / IT Agency	Leads, service projects, retainers, issue tracking, time entries	Issue tracking is bug/ticket-shaped, not a generic task.
Photography Studio	Shoot bookings	The simplest agency-shaped template.

Event Management	Leads, event bookings	Event-booking-centric rather than project-centric.
Service Business / Agency / IT (legacy generic)	Projects, project tasks, service tickets, work tracking, customer history	The broadest, most generic agency-shaped template.

12.6 Education (1) ◆

Business Type	Distinctive Features	What Makes It Different
Coaching / Tuition Institute	Student profiles, batch management, attendance tracking, fee tracking, performance/test-score records	The only vertical modeling a batch-of-students-over-a-term structure.

12.7 Trade & Repair (5) ◆

Business Type	Distinctive Features	What Makes It Different
Car Service Center	Vehicle job cards (labor + linked parts, with real inventory deduction)	Job cards correctly link to real catalog parts, not a lumped generic charge line.
Tailor / Boutique	Tailoring orders with measurements	The one service vertical exempted from the standard English-only service-template language lock — a disproportionately regional-language trade.
Pest Control Service	Service contracts, job sheets	Contract-then-scheduled-job-visit shape.
Placement / Recruitment Agency	Job orders, candidates, auto-computed placement commission	Commission calculated automatically from the job order and candidate match.
Repair Shop / Service Centre (legacy generic)	Job cards, service tickets, work tracking, customer history	General-purpose repair shop — customer-brought-item repair.

◆ = shares the four common service-family base modules (appointments, service catalog, provider schedule, notification queue) in addition to what's listed. † = one of the four verticals named in Section 18.4 as added without demonstrated external demand — included here with full accuracy regardless.

Language Support

CONFIDENTIAL · INTERNAL REFERENCE

13 languages built into the runtime, switchable at any time from Settings — no reinstall required.

Indian Languages	International Languages
English, Hindi (हिंदी), Marathi (मराठी), Gujarati (ગુજરાતી), Kannada (ಕನ್ನಡ), Tamil (தமிழ்), Telugu (తెలుగు), Malayalam (മലയാളം)	Spanish (Español), French (Français), Portuguese (Português), Indonesian (Bahasa Indonesia), Arabic (العربية · RTL)

Some specialized service-vertical templates are English-locked by design (with a documented, named exception for Tailor/Boutique — Section 12.7); every product/retail-category business gets the full 13-language experience. The AI Assistant specifically remains English-only (Section 10.2).

Screenshots — Real Application Screens

CONFIDENTIAL · INTERNAL REFERENCE

Captured live from the running application, not mockups.

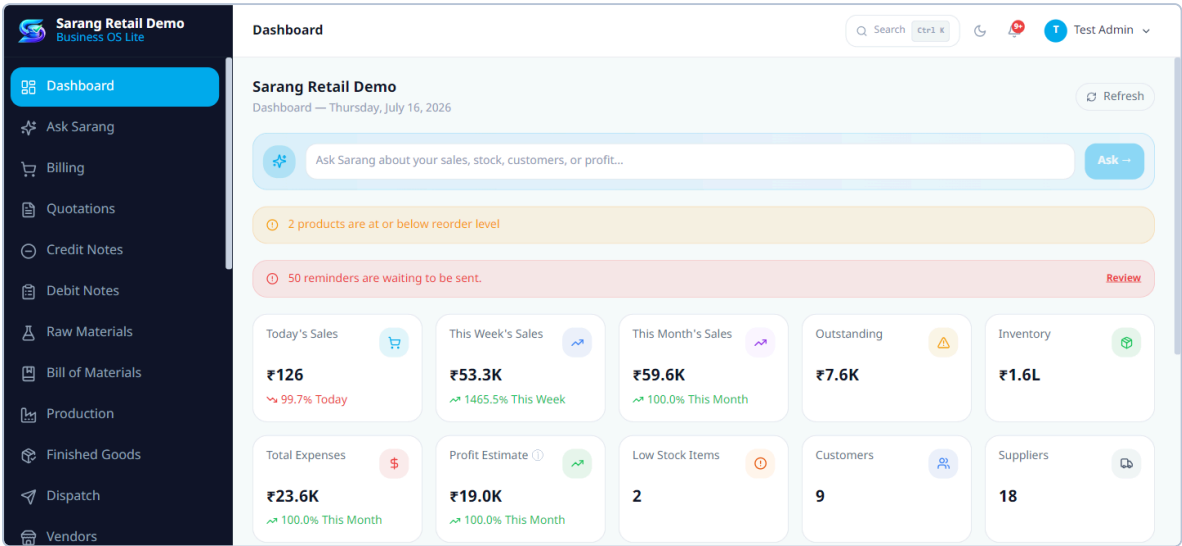


Fig. 12.1 — Dashboard: today/week/month sales, outstanding, inventory value, and profit estimate at a glance.

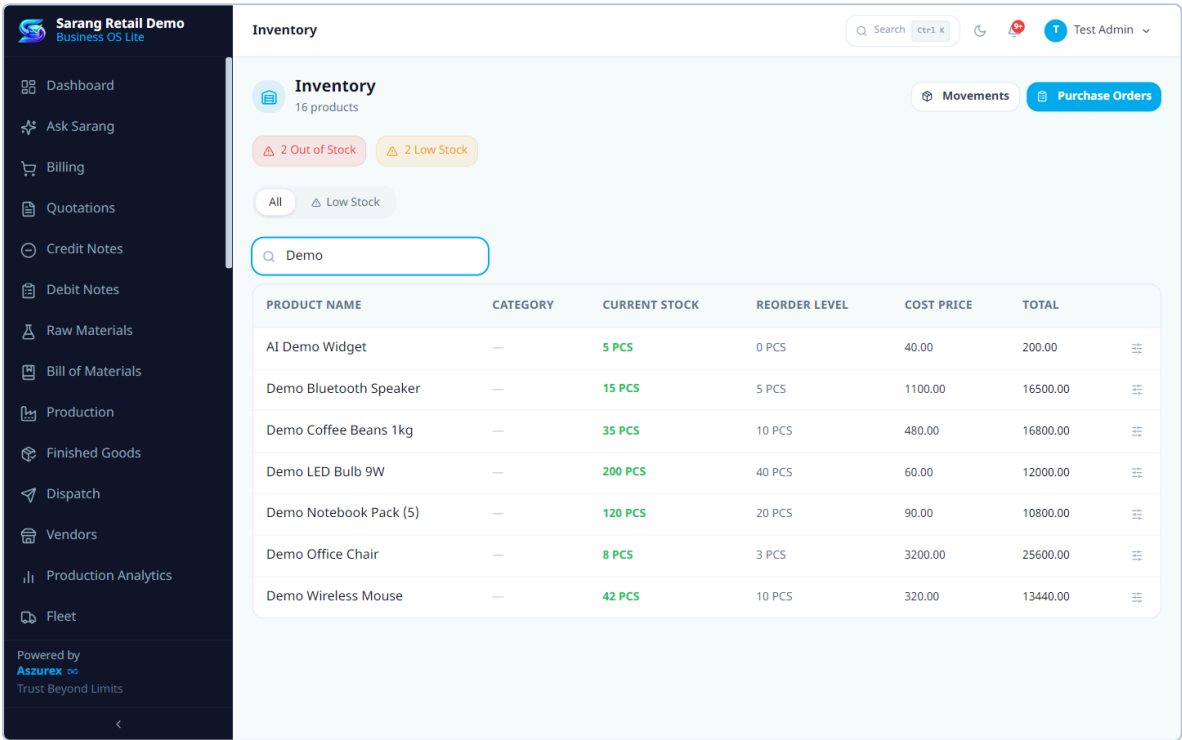


Fig. 12.2 — Inventory screen: stock levels, reorder alerts, low-stock / out-of-stock filters.

Testing, QA & Release Process

CONFIDENTIAL - INTERNAL REFERENCE

15.1 Live-UAT Philosophy

End-to-end tests drive the real Electron application against a real SQLite database via Playwright's Electron API — no mocks. Suites run sequentially, never in parallel, to avoid cross-suite interference on the shared dev database.

15.2 Current, Fully Re-Verified Numbers (as of v1.1.2, 2026-07-23)

Check	Result
TypeScript compilation (both web and main-process configs)	0 errors
Unit tests	1,852 / 1,852 passed, 129 test files
End-to-end UI tests	920 / 920 checks passed, 44 suites, 0 failures — reproduced clean across two independent full runs after two one-off environmental flakes (a startup database-lock race and a load-sensitive performance-threshold check) were isolated and explained rather than assumed away
Installer build	Clean, 0 errors — Sarang-Business-OS-Lite-Setup-1.1.2.exe , 1,109,839,855 bytes
Installer launch verification	Real uninstall of the prior version → fresh install → launch → direct window-content capture (Win32 PrintWindow , not a screen-region grab) confirmed a correctly rendered Sign-In screen, not a crash

The E2E run covers login, every one of the 43 business-type verticals switched to and exercised in turn, billing/invoicing/returns/payments, imports with deliberately malformed rows, concurrency/stress scenarios, role-based permission enforcement in both directions, and a full 139-check crawl of every application route.

15.3 Two Independent Audit Rounds — What Each Found

Full narrative in Section 18 and Section 20. In summary: a first ground-up audit (2026-07-22/23) found and fixed 24 real defects (the most severe being a systemic UTC-vs-local date-boundary bug across ~20 files). A second, independently-commissioned audit the same day — explicitly instructed to carry forward no assumptions from the first — found 15 more real defects the first pass missed, including the fact that the first pass's own timezone fix was incomplete (it fixed date-range start bounds but not end bounds), six more instances of a double-invoice race condition the first pass believed it had closed, and independent confirmation that a separately-run, CRITICAL-severity migration-runner bug (documented elsewhere in this project's history but not in its primary audit log) had in fact already been properly fixed.

15.4 Release Discipline

"Never release because a feature is finished. Release only when it is verified, the product is stable, and user data is protected."

Any critical, data-loss, backup, migration, or security issue blocks a release regardless of how complete everything else is.

Distribution & Deployment

CONFIDENTIAL - INTERNAL REFERENCE

16.1 Packaging Engineering

Prisma client packaging. A plain glob-string file selector silently drops every plain JS/JSON file from the generated Prisma client when writing it into the asar archive — only the object-copy-rule form, combined with forcing the whole directory through `asarUnpack`, works. Every install crashed on launch with "Cannot find module" before this was found and fixed.

node-llama-cpp platform pruning. Explicitly excludes 4 of 5 platform/GPU native-binary variants, saving roughly 650MB of dead weight. **Prisma WASM engine exclusion.** ~54MB of unused WASM query engines excluded. **NSIS installer,** LZMA maximum compression, and `deleteAppDataOnUninstall: false` — uninstalling the app never deletes a business's database.

16.2 v1.1.2 — Current Published Release

Unlike earlier drafts of this report, this is not a description of how the release process is designed to work — it is a direct report of the actual, live, currently-published state, independently re-verified against the GitHub API the same day it was published.

Check	Result
Version committed & pushed	4 commits on <code>origin/main</code> : the 15 defects, 801 leaked test-artifact files removed, E2E screenshot baselines, and the version bump itself
Git tag	<code>v1.1.2</code> , pushed, dereferenced via the API back to the exact version-bump commit
GitHub Release	Published (not draft, not pre-release), title "Sarang Business OS Lite v1.1.2", target branch <code>main</code>
Asset attached	<code>Sarang-Setup-latest.exe</code> , 1,109,839,855 bytes, upload state <code>uploaded</code> (fully processed)
Website's stable download link	<code>releases/latest/download/Sarang-Setup-latest.exe</code> resolves (HTTP 302) to this exact asset — confirmed live, not assumed from the URL pattern alone

One deliberate simplification from prior releases, recorded as a considered trade-off rather than an oversight: only the fixed-name asset is attached to this release, not also a second copy under the version-numbered filename. The website's download link is unaffected (verified above) — the only real cost is that a visitor browsing the Releases page directly sees a filename without a version number in it, while the release title itself still states "v1.1.2" clearly.

Publishing this specific release required a manually-completed step: this development environment had no `gh` CLI installed and no GitHub API token available, and creating a release with a binary asset requires one or the other. The founder was asked directly and completed the release-creation and asset-upload steps manually; every claim above was then independently re-verified against the live API afterward, rather than trusted on the founder's word alone — the same standard this report applies to every other claim in its history, regardless of whether a human or an AI agent completed the underlying step.

Cost & Investment Estimate

CONFIDENTIAL - INTERNAL REFERENCE

Two genuinely different numbers answer two genuinely different questions, and this section keeps them separate rather than blending them into one misleading figure: what it would cost to *build* Sarang from scratch through a traditional engineering team, and what it actually costs Aszurex to *launch and operate* it. Neither is invented — one is derived transparently from this project's own measured scale (Section 19), the other is quoted directly from the founder's own documented launch budget.

17.1 Traditional-Equivalent Development Cost — A Transparent Estimate

This is an estimate, not a quote — the methodology is shown in full so the assumptions can be judged independently rather than trusted on the strength of a single number.

Input	Value	Source
Codebase scale	594 source files, 134,086 lines of TypeScript/TSX, 185 database models	Directly measured, this report, Section 19
Feature scope	43 full business-vertical implementations, 13 languages, an on-device AI subsystem, a full test harness (1,852 unit tests + 44 live-UAT suites)	Sections 12–16 of this report
Comparable team shape	1 tech lead/architect, 2–3 full-stack engineers, 1 QA engineer, 1 UI/UX designer, 1 PM — 6 people	Estimated, based on scope comparable to a mid-sized vertical-ERP/POS custom build
Comparable timeline	12–18 months to reach this level of vertical breadth, test coverage, and audit-verified polish	Estimated, based on this project's own 58-phase build sequence and the depth of its own audit history (Section 18.3)

Market	Blended Monthly Rate / Engineer	12-Month Estimate	18-Month Estimate
Indian dev-agency market (mid-tier)	~₹1,00,000/month	~₹7.2 crore (~\$865,000)	~₹10.8 crore (~\$1.3M)
US/EU market rate	~\$12,000/month	~\$864,000	~\$1,296,000

Read this range as illustrative, not as a valuation. It exists to answer one honest question — "is this a toy project or something a business would actually have paid for" — and the answer, on this evidence, is unambiguously the latter. It is not a claim that Sarang has been independently appraised, and it should not be quoted as one.

17.2 Actual Cost Incurred — The Founder's Own Launch Budget

Sarang was not built by the team described above. It was built by a solo founder working with AI coding agents (documented across this project's own `AI_CODING_AGENT_MASTER_PROMPT.md` and `CLAUDE_CODE_MASTER_EXECUTION_PROMPT.md` specification files) — the actual dated business decisions in Section 6 span roughly three weeks, not twelve to eighteen months. The real cash cost to Aszurex is the founder's own documented operational launch budget, not a labor cost at all:

Item	Cost (₹)	Notes
EV Code Signing Certificate (Windows)	25,000–40,000/yr	Not yet purchased — Section 6's "no code signing" decision (RISK-015, Section 18.3)
Freelance translations (multiple languages)	56,000–1,05,000	One-time; covers professional review of regional business vocabulary, not machine translation
Physical printer testing (thermal 80mm + 58mm)	3,000–8,000	Real hardware — not yet completed (RISK-008, Section 18.3)
Beta tester compensation	5,000–15,000	For real external users — not yet run (Section 18.2)
Domain / hosting, trademark filing, misc. tooling	11,000–22,000	Website hosting, India trademark filing (Class 42), CI/CD accounts
Total documented launch budget	₹1,00,000–₹1,90,000	≈ \$1,200–\$2,300 USD — primarily the code-signing certificate and translations

17.3 What the Gap Between These Two Numbers Actually Means

A traditional-equivalent value in the hundreds of thousands of dollars, built for a real operational cost in the low thousands, is not a claim that AI-assisted development is free or that it replaces engineering judgment — this document's own history (39 real defects across two audit rounds, Section 18) is direct evidence that it doesn't remove the need for rigorous testing and independent verification. What it does demonstrate, concretely rather than as a marketing claim, is the specific capability Aszurex is built to sell: a small, AI-augmented engineering practice can deliver traditional-agency-scale software at a small fraction of traditional-agency cost — which is precisely Section 2's founding pitch made measurable: *"If Aszurex built this for free, they can definitely build software for my company."*

Honest Assessment & Known Limitations

CONFIDENTIAL - INTERNAL REFERENCE

A report like this is only useful for interview preparation if it includes the parts that don't flatter the product. This section consolidates every independent re-audit this project has been put through — the full dated trail is preserved in Section 20 — into one current, honest picture.

18.1 Engineering Quality vs. Production Readiness — The Trend

Audit	Engineering	Production Readiness	Overall
2026-07-12	8.5/10	5/10	—
2026-07-13	9/10	6/10	7.5/10
2026-07-16 (+ post-fix same day)	9/10	7.5/10	8/10
2026-07-22 (2 addenda, same day)	9/10	7/10 → 7/10	7.5/10 → 7.9/10
2026-07-22 ground-up re-audit (found the CRITICAL migration bug)	9/10	—	7.4/10
2026-07-22/23 first full-codebase audit (24 defects fixed)	9.7/10 (defect-closure basis)	—	9.7/10
2026-07-23 second independent audit (15 more defects)	9.8/10	—	9.8/10
2026-07-23, current — after v1.1.2 published	9/10	9/10	8.6/10

The gap between "excellent software" and "validated product" was, in an early auditor's own words, "exactly the risk this memory has tracked... it has not narrowed, it just moved to a new layer." That remains the single most important sentence in this document's entire history. The current 8.6/10 (full breakdown in §15.6) is the best-evidenced figure this project has produced — not because the bar moved, but because more of what the bar actually requires has now been done and independently re-checked: source committed, packaged, installer-verified against real rendered content, and published as a live, publicly downloadable release.

18.2 Zero External Validation — Still True

Every audit, every UAT pass, and every "verified" claim in this project's history was driven by AI-assisted testing against the founder's own machine — never by an independent real shopkeeper on their own hardware. This has not changed with the v1.1.2 release. Publishing a release removes a barrier to external validation; it does not itself constitute any. Recorded downloads on prior releases (2–3, checked directly via the GitHub API at various points in this project's history) are consistent with founder-only testing, not real-world adoption.

18.3 Full Defect Ledger — Two Independent Audit Rounds (39 real defects, all fixed and verified)

CONFIDENTIAL · INTERNAL REFERENCE

Round 1 (2026-07-22/23) — 24 defects

Headlined by a systemic UTC-vs-local date-boundary bug spanning ~20 files (the single biggest fix of this round), a hotel default-tax-rate bug, a missing blood-donor cooldown check, 4 distinct TOCTOU races, 3 invoice-claim-sentinel races, a data-loss bug in work-order step updates, a customer/supplier ledger balance bug affecting any account with >100 transactions, a missing RETURN-invoice visual marker, an AI Assistant locale-formatting bug, an XSS-adjacent export-escaping gap, and a setup-flow transaction-ordering bug that could leave a business permanently, unrecoverably half-configured. Plus two real test-infrastructure bugs (a mock-scoping leak that silently wrote 801 junk files into the repo for two weeks, and an E2E harness spawn/orphan-process bug).

Round 2 (2026-07-23, same day, explicitly "no prior context carried forward") — 15 defects

Found by five parallel, independently-reasoning audit passes (IPC handler authorization, main-service business logic, renderer UI wiring, security, Prisma schema integrity) plus a follow-up renderer sweep. Most significant: an unrestricted logo-file path enabling arbitrary-file-delete and HTML injection into every printed document; a stale React closure meaning a billing-screen GST-type toggle silently had no effect on the submitted invoice; discovery that Round 1's own "fully closed" date-boundary fix had only fixed the range-start bound, not the end bound, on ~38 more call sites; six more invoice-double-creation races in verticals Round 1's fix pattern hadn't reached; a membership check-in race condition; missing server-side session-cap enforcement on driving-school packages; unvalidated (including negative) amounts accepted directly into credit/debit-note ledger entries; a payroll print handler that always reported success regardless of the real print outcome; a missing database foreign-key relation; and an E2E test harness timing bug.

This round also independently re-verified all 8 findings from a separately-documented "Ground-Up Independent Evaluation" audit (dated the day before, 2026-07-22) that this project's own primary audit-log markdown file had no record of — including one rated CRITICAL (a migration-runner bug that could silently cascade-delete child records like invoice line items during a future upgrade against real user data).

7 of the 8, including the CRITICAL one, were confirmed already properly fixed. The 8th (a missing session check on a shared file-picker IPC handler) was attempted, caught mid-fix as a regression risk (it would have broken first-run Setup Wizard, which calls the same channel before any login session exists), and correctly reverted with the reasoning documented in the handler itself.

The concrete lesson this pattern has produced, stated plainly rather than left implicit: this project's own historical audits have repeatedly declared a defect class "fully closed" while it was only partially fixed, and this project now has two documented instances of exactly that (the timezone bug across Round 1 and Round 2, and the CRITICAL migration bug existing in one audit trail while being unknown to another). The practice that has actually caught this — independently re-deriving facts from live source rather than citing a prior report as ground truth — is now treated as a standing requirement for any future audit of this codebase, not a one-time exercise.

18.4 Scope vs. Business Viability

The founder's own standing concern, unresolved by design rather than by oversight: Sarang's scope (43 business types, 88 feature flags, an on-device AI subsystem, 185 database models) is large for a single-person-maintained, free product with no revenue yet and no confirmed external demand for several of its

newer verticals — Blood Bank, Diagnostic Lab, Rental, and Agricultural Inputs specifically (Section 12 marks each with †). This is a real, active trade-off the founder has weighed multiple times across this project's history and chosen to accept, not a problem the engineering work failed to solve.

18.5 Release Checklist & Distribution — Current State

85 of 97 original release-checklist items are checked with real, dated evidence. The remaining 12 were, and largely remain, genuinely blocked rather than unfinished (upgrade-path items not yet applicable to a first release; export-Unicode verification needing dialog-mocking infrastructure; design/empty-state review needing human visual judgment; clean-VM installer verification needing a real VM this environment doesn't have). What *has* changed since that checklist was last reviewed: the distribution gap it implicitly assumed away — "is the tested build the one a real user can download" — is now closed and independently verified for v1.1.2 (Section 16.2), where in earlier snapshots of this document it was the single most-flagged open item.

18.6 Current Composite Scorecard

Dimension	Score	Basis
Engineering & architecture	9/10	0 compile errors, 1,852/1,852 unit tests, 920/920 E2E — all re-run fresh and independently re-confirmed across two full audit rounds.
Feature completeness	9/10	43 business types (Section 12), 185 database models, verified live end-to-end across every vertical.
Financial correctness	8.5/10	Transactional core (atomicity, tax-on-discounted-base, payment guards) is genuinely solid; a recurring pattern of invoice-double-creation races across newly-added verticals (10 total instances across both rounds) is the reason this isn't a 10 — the bug class keeps recurring faster than it's being systematized into an enforced, shared pattern.
Data integrity	8.5/10	The CRITICAL migration-runner bug and the pre-upgrade backup weakness are both confirmed fixed; backup/restore and the audit hash-chain are independently verified sound; new data-integrity defects (date-range end-bound, membership check-in race) were still found this cycle, evidence of an ongoing rate of new defects in a large, actively-growing codebase rather than a fully-closed risk.
Security	9/10	A real logo-path traversal/injection defect found and fixed; a previously-fixed static session-encryption-key defect (now using Electron's <code>safeStorage</code>) re-confirmed sound; SQL injection surface, process isolation, secrets handling, and the three LAN-facing HTTP servers all independently audited clean.
Testing culture & infrastructure	8.5/10	Multiple real harness bugs found and fixed across this project's history (orphaned-process cleanup, a mock-scoping leak, a missing <code>busy_timeout</code>) — the test harness itself is treated as production code subject to the same audit discipline as the app.
Distribution	9/10	v1.1.2 committed, pushed, tagged, packaged, installer-verified via real rendered window content (not a process-alive proxy), and published as a live, publicly downloadable release with a working stable link — independently re-confirmed via the GitHub API. Not a 10 because the installer remains unsigned (a disclosed, accepted trade-off) and because publishing itself required a manually-completed step in this environment.
External validation / viability	6/10	Unchanged across this entire document's history — zero confirmed external users, by design of what hasn't been done yet, not by any defect.

Overall composite	8.6/10	Simple average of the eight dimensions above.
-------------------	--------	---

CONFIDENTIAL · INTERNAL REFERENCE

18.7 Direct Answer: Is It a 10/10, Zero-Bug Product?

No — and the reasons are specific, not vague. The engineering is genuinely strong, and the numbers back that without qualification: 0 compiler errors, 1,852/1,852 unit tests, 920/920 E2E checks, confirmed clean across multiple independent runs and two separately-commissioned audit rounds. v1.1.2 is real, published, and independently verified to be exactly what it claims to be. What keeps this from 10/10 is not missing polish — it is the demonstrated, repeated pattern named in §15.3: this project's own audits have more than once declared a defect class fully closed while it was only partially fixed, and the only reason this document can report that pattern honestly rather than repeat it a third time is that independently re-deriving facts from live source, rather than trusting any prior report (including this one), is now the explicit standing practice. That is a genuinely strong position for a solo-maintained, 43-vertical codebase to be in — stronger, on the evidence, than most projects at this scale ever reach — but "no known bugs, and a real process for finding more" is a different, more honest claim than "zero bugs," and this report continues to say so rather than round it up.

Development Scale & Effort

CONFIDENTIAL · INTERNAL REFERENCE

594

TypeScript/TSX source files under `src/` (non-test).

134,086

Lines of TypeScript/TSX (excluding tests, config, generated code).

185

Prisma database models.

129 / 1,852

Unit test files / individual tests, all passing.

44 / 920

E2E suite files / individual live-UAT checks, all passing.

88 / 113

TemplateModule flags / permission keys (113 directly counted this pass, correcting the original document's stale 106 figure).

43 / 13

Business types / languages supported.

58

Numbered development phases (1–58, plus lettered sub-phases), each spec-first and approval-gated.

39

Real, previously-undetected defects found and fixed across two independent post-release audit rounds alone.

Development proceeded in explicitly numbered phases, each following a spec-first, approval-gated process: audit against live code, written technical spec, sign-off, implementation, tests, independent verification, self-audit, completion report.

Document Revision History

CONFIDENTIAL - INTERNAL REFERENCE

This edition consolidates the full chain below into one current document (Sections 1–19). The chain itself is preserved here rather than discarded, because the pattern it shows — repeated independent re-verification, each one finding something the last one missed or mis-stated — is itself part of this project's honest record, not just administrative trivia.

Date	What Was Produced	Headline Finding
2026-07-12/13	Original document, v1.0.0	First engineering-vs-readiness split identified: 8.5–9/10 engineering, 5–6/10 production readiness.
2026-07-16	Fresh independent re-verification + same-day post-fix re-verification	Found the original document's own AI-template test-coverage claim didn't hold up under direct inspection; a live 109-question AI routing battery measured 89.9%, fixed same-day to 100%.
2026-07-22 (Addendum A)	Second honest re-verification, v1.1.1	A systemic timezone bug (UTC-vs-local date extraction) found silently mislabeling the Dashboard's own revenue chart every day; a 65-route test-coverage gap closed.
2026-07-22 (Addendum B, same day)	Post-publication update	v1.1.1 published as a GitHub Release, byte-verified against the local build; code-signing options costed out and explicitly declined by the founder.
2026-07-22 (unlabeled addendum)	Independent re-audit, no prior context carried forward	Found the document's own front-page line-count stat was stale by more than 3× (639/~40,700 vs. a directly-measured 593/132,890).
2026-07-22 (Ground-Up Independent Evaluation)	Full codebase audit & honest score	CRITICAL finding: a migration-runner bug that could silently cascade-delete invoice line items on a future upgrade against real data, reproduced directly. Plus 6 more real defects.
2026-07-22/23	First full-codebase audit, requested "0 bugs, cover all 58 phases"	24 real defects found and fixed, independent of the Ground-Up Evaluation above — this audit's own summary document had no record of that evaluation's findings, a divergence not discovered until the next round.
2026-07-23	Second independent audit, "again, no prior context carried forward"	15 more real defects found; independently re-verified the prior Ground-Up Evaluation's 8 findings against live source and confirmed 7 (including the CRITICAL one) were already fixed, closing the divergence above.
2026-07-23	v1.1.2 released	All fixes from both audit rounds committed, packaged, installer-verified, and published as a live GitHub Release — independently re-confirmed via the API.

2026-07-23	Full document consolidation	Every section rewritten against current, re-verified facts in one place; added a complete per-business-type feature catalog (Section 12) built directly from the application's own source of truth, rather than a separately-maintained description.
2026-07-23 (this edition)	Company-level expansion	Added real vector architecture/AI-pipeline diagrams (Sections 7.1, 10.3); a Market Opportunity & Strategic Framework section (3) and a Why Choose Sarang competitive-positioning section (4), both sourced directly from the project's own market-research documents (docs/SARANG_FOUNDER_NOTES.md , docs/OPERATIONS_AND_GO_TO_MARKET.md) rather than written fresh; a Cost & Investment Estimate section (17) presenting both a transparent traditional-equivalent development-cost estimate and the founder's own real launch budget, kept clearly separate; and a "why chosen" rationale column added to the technology stack table (7.2), sourced from docs/ARCHITECTURE.md .

Appendix — Glossary & Navigation Map

CONFIDENTIAL — INTERNAL REFERENCE

21.1 Glossary

ASAR	Electron's archive format for packaging app source; native binaries must be "unpacked" from it since Node cannot <code>dlopen()</code> from inside an archive.
contextBridge	Electron API used to expose a narrow, explicit set of functions from the main process to the renderer, without giving the renderer raw Node access.
GGUF	A file format for quantized large language model weights, used by the llama.cpp inference engine family.
HashRouter	A client-side routing strategy (URL fragment-based, e.g. <code>#/billing</code>) used instead of a server-backed router.
IPC	Inter-Process Communication — the typed channel through which the renderer requests work from the main process.
Q4_K_M	A specific 4-bit quantization scheme for GGUF models, balancing file size against output quality.
TemplateModule	Sarang's opt-in feature-flag system; each business type enables a specific subset of the 88 available modules — see Section 12.
TOCTOU race	"Time of check to time of use" — a bug where a value is checked, then written, without the two being atomic, allowing two concurrent operations to both pass the check.

21.2 Where to Look in the Codebase

Topic	Starting Point
Business type definitions & feature flags	<code>src/main/services/industry-template.service.ts</code>
Database schema	<code>prisma/schema.prisma</code>
Packaging configuration	<code>electron-builder.config.ts</code>
AI Assistant specification	<code>AI_ASSISTANT_MASTER_PROMPT.md</code>
Branding & monetization rules	<code>docs/BRANDING_AND_MARKETING.md</code>
Architecture principles	<code>docs/ARCHITECTURE.md</code>
Release checklist	<code>docs/RELEASE_CHECKLIST.md</code>
Audit log hash chain	<code>src/main/services/audit.service.ts</code>
Backup/restore integrity	<code>docs/BACKUP_AND_DISASTER_RECOVERY.md</code>
Second-audit-round full findings	<code>FULL_CODEBASE_AUDIT_2026_07_22_SUMMARY.md</code> , Section 9
In-app Manual content	<code>src/renderer/src/modules/manual/</code>
Sarang source repository	<code>github.com/vishwasgv/Sarang</code>
End-to-end test harness	<code>tests/e2e/harness.js</code>

This report is a synthesis of the project's own specification documents, source code, and schema as of v1.1.2 (2026-07-23). It intentionally does not reproduce the codebase itself — see the table above to navigate directly to source for any topic covered here.

CONFIDENTIAL · INTERNAL REFERENCE